평가자 일치도 분석 (Inter-rater Reliability)

LLM-as-Judge 여러 모델이 얼마나 같은 판단을 내리는지 수치로 증명하기

LLM-as-Judge 평가 시스템이란?

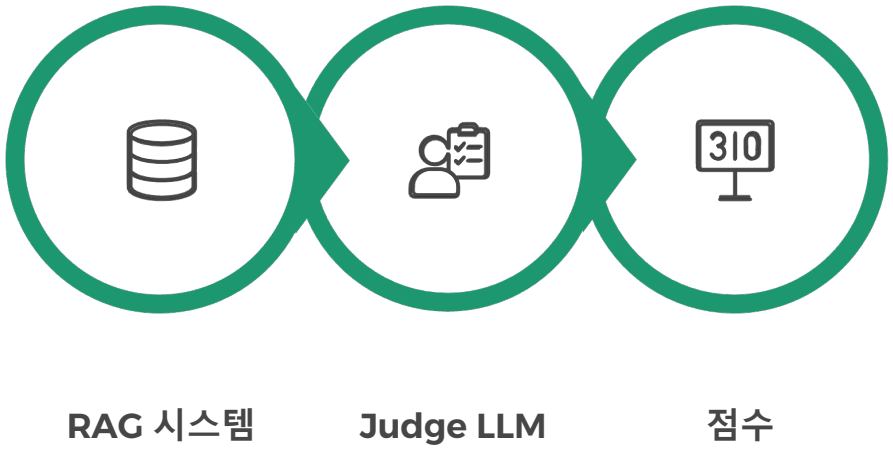
AI가 AI를 채점하는 구조

사람이 직접 채점하기엔 데이터가 너무 많을 때 사용합니다.

💡 **쉽게 말하면:** 학교 시험에서 선생님 한 명이 300명 답안지를 채점하는 것은 힘듭니다. **LLM-as-Judge**는 채점 기준(프롬프트)을 AI에게 주고 AI가 자동으로 채점하게 하는 것입니다.

- **Input (입력):** 질문 + RAG 시스템 답변 + 정답
- **Judge (채점자):** GPT-4.1-mini, Gemini 등 LLM
- **Output (결과):** 0.0 / 0.5 / 1.0 점수

흐름 요약



왜 여러 Judge를 사용하나요?

단일 Judge의 한계와 다중 Judge의 필요성

문제	해결책
AI 하나의 채점은 편향될 수 있다	여러 AI Judge로 채점해 비교
"이 AI가 신뢰할 수 있나?" 증명이 어렵다	Judge 간 일치도(Kappa)로 수치 증명
모델마다 기준이 달라질 수 있다	동일한 프롬프트 + 동일한 기준 부여

핵심: 여러 Judge가 "같은 답변에 같은 점수"를 준다면, 그 평가 시스템은 신뢰할 수 있습니다.

평가자 일치도(Inter-rater Reliability)란?

여러 채점자가 같은 대상을 얼마나 일관되게 평가하는지 측정하는 지표입니다.

💡 쉽게 말하면

심사위원 3명이 같은 노래를 듣고 각각 점수를 줄 때,

- 세 명이 매번 같은 순위를 매긴다면 → "일치도가 높다"
- 세 명이 전혀 다른 순위를 매긴다면 → "일치도가 낮다"

LLM-as-Judge에서도 마찬가지입니다. Judge 세 개가 같은 답변에 같은 점수를 주면 → 신뢰할 수 있는 평가 시스템

일치도가 낮으면 어떤 문제가 있나요?

상황	결과
Judge A: 1.0점, Judge B: 0.0점 (같은 답변)	평가 결과를 믿을 수 없음
프로젝트 회의에서 평가 결과 발표	"이 AI가 왜 저 AI랑 다르게 채점하나요?" 질문
A/B 테스트에서 모델 성능 비교	채점자마다 결과가 달라 결론 도출 불가

일치도 측정 방법 비교

단순 일치율부터 Fleiss' Kappa까지

방법	채점자 수	특징
단순 일치율 (Percent Agreement)	2명 이상	쉽지만 우연 일치를 보정 안 함
Cohen's Kappa	정확히 2명	우연 일치를 제거한 정밀한 지표
Fleiss' Kappa	3명 이상	Cohen's Kappa의 일반화 버전

핵심: Cohen's Kappa는 쌍(pair)별로, Fleiss' Kappa는 세 Judge 전체 동시 측정에 사용합니다.

단순 일치율의 함정 — 왜 보정이 필요한가?

100% 일치해도 믿을 수 없는 경우가 있습니다.

💡 쉽게 말하면

두 채점자에게 100개 답변을 채점시켰더니 98개가 일치했습니다. 그런데 이 98개가 "둘 다 1.0점을 줬기 때문"이라면? 두 채점자가 **아무 답변에나 무조건 1.0점**을 눌렀어도 98%가 일치합니다.

이렇게 **우연히 일치하는 부분을 제거**해야 진짜 일치도를 알 수 있습니다.

왜 33%가 우연 일치인가?

점수가 0.0 / 0.5 / 1.0 세 가지라면:

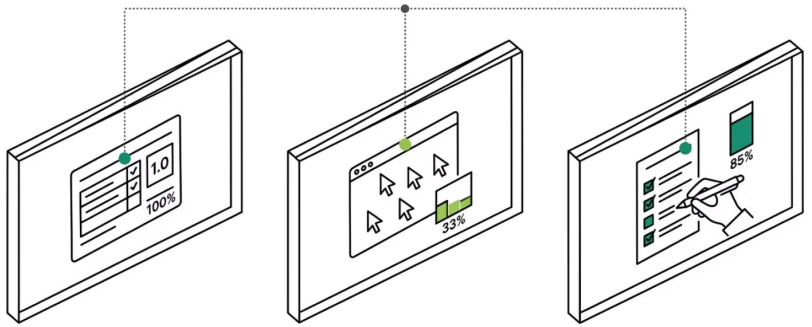
- 채점자 A가 무작위로 고를 확률: 각 1/3
- 채점자 B가 같은 값 무작위로 고를 확률: 1/3
- → 우연 일치 기대 확률 = 1/3 × 1/3 × 3 = 1/3 ≈ 33%

결론: 단순 일치율에서 이 우연 일치분(33%)을 빼고 나서야 진짜 일치도를 알 수 있습니다. 이것이 바로 **Cohen's Kappa**가 하는 일입니다.

단순 일치율 vs 진짜 일치 – 구체적인 예시

같은 일치율이라도 의미가 전혀 다릅니다.

상황	단순 일치율	실제 의미
두 채점자 모두 "항상 1.0점"	100%	실제로는 판단 안 함
두 채점자 무작위 클릭 (점수 3가지)	~33%	그냥 랜덤
두 채점자가 기준대로 신중하게 채점	85%	진짜 일치



항상 1.0점
(100% 일치, 실제 판단 없음)

무작위 클릭
(33% 일치, 랜덤)

기준대로 신중하게 채점
(85% 일치, 진짜 일치)

Cohen's Kappa — 수식과 해석

Cohen's Kappa(κ)는 두 평가자 간 일치도를 **우연 일치를 제거**하고 측정하는 지표입니다.

수식

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

기호	이름	뜻
P_o	Observed agreement (관측 일치율)	실제로 두 채점자가 같은 점수를 준 비율
P_e	Expected agreement (기대 일치율)	무작위로 채점해도 우연히 일치할 비율
1 - P_e	최대 가능한 보정 후 일치율	우연을 제거한 후 남은 최대값

💡 수식을 쉽게 말하면

κ = (실제 일치 - 우연 일치) ÷ (완전 일치 - 우연 일치)

- 분자: "우연을 넘어서 진짜로 일치한 부분"
- 분모: "우연을 완전히 넘어섰을 때의 최대값"
- 결과: 0이면 "우연 수준", 1이면 "완벽 일치"

P_e (우연 일치율) 계산 방법

점수가 0.0 / 0.5 / 1.0이고, 두 채점자 A, B의 점수 분포가 아래와 같을 때:

채점자 A: 0.0점 40개, 0.5점 30개, 1.0점 30개 (전체 100개)
채점자 B: 0.0점 35개, 0.5점 25개, 1.0점 40개 (전체 100개)
P_e = (40/100 × 35/100) + (30/100 × 25/100) + (30/100 × 40/100)
= 0.14 + 0.075 + 0.12 = 0.335 (약 33.5% 우연 일치)

Kappa 값 해석 기준

κ 값에 따라 평가 시스템의 신뢰도를 판단합니다.

κ 값	해석	실제 의미
< 0.00	일치 없음 (Poor)	우연보다도 덜 일치 — 뭔가 잘못됨
0.00 ~ 0.20	매우 낮음 (Slight)	거의 우연 수준
0.21 ~ 0.40	낮음 (Fair)	약간의 일치 — 개선 필요
0.41 ~ 0.60	**보통 (Moderate)**	**실용적인 최소 기준선**
0.61 ~ 0.80	높음 (Substantial)	신뢰할 수 있는 평가 시스템
0.81 ~ 1.00	거의 완벽 (Almost perfect)	매우 일관된 채점자들

참고: LLM-as-Judge 시스템에서 $\kappa \geq 0.61$이면 "이 평가 시스템은 신뢰할 수 있다"고 보고할 수 있습니다.

Cohen's Kappa 계산 — 단계별 예시

5개 샘플로 직접 계산해봅니다.

두 Judge의 채점 결과

샘플	Judge A	Judge B	일치?
1	1.0	1.0	✓
2	0.5	0.0	✗
3	1.0	1.0	✓
4	0.0	0.0	✓
5	0.5	1.0	✗

P_o (관측 일치율): 3/5 = 0.60

P_e 계산

Judge A 분포: 0.0점→1개, 0.5점→2개, 1.0점→2개
Judge B 분포: 0.0점→1개, 0.5점→0개, 1.0점→4개
P_e = (1/5×1/5)+(2/5×0/5)+(2/5×4/5)
= 0.04+0.00+0.32 = 0.36

κ 계산

κ = (0.60 - 0.36) / (1 - 0.36)
= 0.24 / 0.64 = 0.375
→ "낮음 (Fair)" 해석

해석: κ = 0.375는 "낮음(Fair)" 수준입니다. 평가 프롬프트를 개선하거나 0.5점 기준을 명확히 해야 합니다.

Cohen's Kappa — 실제 코드 (scikit-learn)

Python으로 직접 계산하는 방법

```
from sklearn.metrics import cohen_kappa_score

# 점수를 문자열 레이블로 변환 (이산형 처리)
labels_a = ["1.0", "0.5", "1.0", "0.0", "0.5"]
labels_b = ["1.0", "0.0", "1.0", "0.0", "1.0"]

kappa = cohen_kappa_score(labels_a, labels_b)
print(f"κ = {kappa:.4f}")  # → κ = 0.3750
```

💡 왜 float가 아닌 문자열(str)로 변환하나요?

cohen_kappa_score는 내부적으로 범주형(categorical) 레이블을 기대합니다. 0.0, 0.5, 1.0은 숫자처럼 보이지만, 여기서는 "등급 이름"입니다. str(score)로 변환하면 함수가 이를 3가지 범주("0.0", "0.5", "1.0")로 인식합니다.

Fleiss' Kappa — 3명 이상 채점자 동시 측정

Fleiss' Kappa는 Cohen's Kappa를 3명 이상으로 확장한 버전입니다.

💡 쉽게 말하면

Cohen's Kappa는 "A vs B", "A vs C", "B vs C"처럼 **두 명씩** 비교합니다. Fleiss' Kappa는 **A, B, C 세 명을 한꺼번에** 비교합니다.

비유: 축구 경기에서 심판 3명의 판정 일치도를 한 번에 계산하는 것.

Cohen's Kappa vs Fleiss' Kappa 비교

항목	Cohen's Kappa	Fleiss' Kappa
채점자 수	정확히 2명	3명 이상
계산 방식	쌍별 비교 (pair-wise)	전체 동시 비교
결과 수	채점자 쌍 수만큼	하나의 값
사용 상황	두 모델/사람 비교	세 개 이상 Judge 한번에 비교

Fleiss' Kappa 수식 개념

우연 일치를 제거한 전체 평균 일치도를 계산합니다.

$$\text{Fleiss' } \kappa = \frac{P_o - P_e}{1 - \bar{P}_e}$$

기호 설명

- P̄_o: 전체 샘플에서 평균 관측 일치율
- P̄_e: 각 범주별 전체 채점 비율로 계산한 기대 일치율

Fleiss' Kappa 수동 계산 예제

범주 빈도 행렬을 바탕으로 수식을 단계별로 직접 풀어봅니다.

변수 정의

- 샘플 수 (N): 3
- 평가자 수 (n): 3
- 범주 수 (k): 3 (0.0점, 0.5점, 1.0점)
- 총 투표 수: N × n = 9

Step 1: 기대 일치율 ($\bar{P}_e$) 계산

각 범주의 전체 투표 비율(p_j)을 구합니다.

범주	총 투표 수	비율 (p_j)
0.0점	2표 (샘플2에서 2표)	2/9
0.5점	2표 (샘플1, 2에서 각 1표)	2/9
1.0점	5표 (샘플1에서 2표, 샘플3에서 3표)	5/9

수식 전개

각 범주 비율의 제곱합으로 기대 일치율을 계산합니다.

$$\bar{P}_e = \sum_{j=1}^{k} p_j^2 = \left(\frac{2}{9}\right)^2 + \left(\frac{2}{9}\right)^2 + \left(\frac{5}{9}\right)^2$$

$$= \frac{4}{81} + \frac{4}{81} + \frac{25}{81} = \frac{33}{81} = \frac{11}{27} \approx 0.4074$$

💡 $\bar{P}_e = 0.4074$ 의미: 평가자들이 완전히 무작위로 채점해도 약 40.7%는 우연히 일치할 것으로 기대됩니다.

Step 2: 관측 일치율 (P̄_o) 계산

각 샘플별 일치도를 구한 뒤 평균을 냅니다.

샘플별 일치도 수식

*n_{ij}*는 i번째 샘플이 j번째 범주에서 받은 투표 수입니다.

$$P_i = \frac{1}{n(n-1)} \sum_{j=1}^{k} (n_{ij}^2 - n_{ij})$$

샘플	투표수 [0.0, 0.5, 1.0]	계산	P_i
1	[0, 1, 2]	(0+0+2)/6	1/3
2	[2, 1, 0]	(2+0+0)/6	1/3
3	[0, 0, 3]	(0+0+6)/6	1

평균 관측 일치율 계산

$$\bar{P}_o = \frac{1}{N} \sum_{i=1}^{N} P_i = \frac{1}{3}\left(\frac{1}{3} + \frac{1}{3} + 1\right)$$

$$= \frac{1}{3} \quad \frac{5}{3} = \frac{5}{9} \approx 0.5556$$

💡 *|bar{P}_o* = 0.5556 의미: 실제로 평가자들이 관측된 일치율은 약 55.6%입니다. 기대 일치율(40.7%)보다 높으므로 우연 이상의 일치가 존재합니다.

Step 3: Fleiss' Kappa (κ) 최종 계산

P̄_o와 P̄_e를 수식에 대입해 최종 κ 값을 구합니다.

수식 대입

$$\kappa = \frac{P_o - P_e}{1 - \bar{P}_e}$$

$$\kappa = \frac{\frac{5}{9} - \frac{11}{27}}{1 - \frac{11}{27}} = \frac{\frac{15}{27} - \frac{11}{27}}{\frac{16}{27}} = \frac{4}{16} = 0.25$$

💡 결과 해석: 계산된 Fleiss' Kappa 값은 κ = 0.25입니다. Landis & Koch 기준에 따르면 0.21 ~ 0.40 구간은 '보통 수준의 일치도(Fair agreement)'에 해당합니다.

검증 코드 (Python)

```
from statsmodels.stats.inter_rater import fleiss_kappa
import numpy as np

# 범주 빈도 행렬 (N=3 샘플, k=3 범주)
# 각 행: [0.0점 투표수, 0.5점 투표수, 1.0점 투표수]
ratings_matrix = np.array([
    [0, 1, 2],  # 샘플 1
    [2, 1, 0],  # 샘플 2
    [0, 0, 3],  # 샘플 3
])

# 이미 빈도 행렬이므로 바로 계산!
fleiss_k = fleiss_kappa(ratings_matrix)
print(f"Fleiss' Kappa: κ = {fleiss_k:.4f}")
```

Fleiss' Kappa — 범주 빈도 행렬과 계산 코드

statsmodels로 직접 계산하는 방법

Fleiss' Kappa는 각 "샘플"마다 채점자들이 어떤 범주에 몇 명씩 투표했는지를 집계합니다. 예를 들어 샘플 1에 대해 Judge 3명이 [1.0, 1.0, 0.5]를 줬으면:

- 1.0점 투표: 2개, 0.5점 투표: 1개 → 이 분포를 전체에 집계

범주 빈도 행렬 (statsmodels 입력 형식)

샘플	0.0점 받은 수	0.5점 받은 수	1.0점 받은 수
샘플 1	0	1	2
샘플 2	2	1	0
샘플 3	0	0	3

참고: 이 표의 각 행 합계 = Judge 수 (3)

Fleiss' Kappa — 계산 코드 (statsmodels)

Python으로 직접 계산하는 방법

원시 데이터 → 범주 빈도 행렬 변환

```
from statsmodels.stats.inter_rater import fleiss_kappa, aggregate_raters
import numpy as np

# 원시 평가 데이터 (N=3 샘플, Judge=3명)
# 각 행: [Judge 1 점수, Judge 2 점수, Judge 3 점수]
raw_ratings = np.array([
    [1.0, 1.0, 0.5],  # 샘플 1
    [0.0, 0.5, 0.0],  # 샘플 2
    [1.0, 1.0, 1.0],  # 샘플 3
])

# 1. 점수를 정수 범주(0, 1, 2)로 매핑
score_map = {0.0: 0, 0.5: 1, 1.0: 2}
int_ratings = np.vectorize(lambda x: score_map.get(x))(raw_ratings)

# 2. 원시 데이터를 범주 빈도 행렬로 변환
table, categories = aggregate_raters(int_ratings)
```

Fleiss' Kappa 계산 및 출력

```
# 3. Fleiss' Kappa 계산
fleiss_k = fleiss_kappa(table)
print(f"Fleiss' Kappa: κ = {fleiss_k:.4f}")
# 출력: Fleiss' Kappa: κ = 0.2500
```

💡 aggregate_raters()는 원시 평가 행렬(샘플 × Judge)을 자동으로 범주 빈도 행렬로 변환해줍니다. 이미 빈도 행렬이 있다면 바로 fleiss_kappa()에 넣으면 됩니다.

이산형 점수가 중요한 이유

왜 0.0 / 0.5 / 1.0처럼 딱 끊어진 점수를 사용하나요?

💡 쉽게 말하면

Judge에게 "0~1 사이 아무 숫자나 주세요"라고 하면 → 0.73, 0.81, 0.69처럼 다양한 숫자

Judge에게 "0.0 / 0.5 / 1.0 중 하나만 고르세요"라고 하면 → 세 가지 중 하나

Kappa 계산은 "같은 범주를 골랐냐"를 기준으로 합니다. 연속형(0.73 vs 0.81)은 "다른 숫자"로 처리되어 일치율이 낮게 나옵니다.

choices 설정 코드

```
from openevals.llm import create_llm_as_judge

judge = create_llm_as_judge(
    prompt=KOREAN_CORRECTNESS_PROMPT,
    feedback_key="correctness",
    model="openai:gpt-4.1-mini",
    choices=[0.0, 0.5, 1.0],  # ← 이산형 제한이 핵심!
)
```

⚠️ **주의:** choices를 설정하면 Judge LLM이 반드시 저 세 값 중 하나만 반환합니다. 설정하지 않으면 Judge가 임의의 숫자를 반환해 Kappa 계산이 무의미해집니다.

이산형 vs 연속형 – 비교와 점수 기준

같은 데이터라도 점수 형식에 따라 Kappa 계산 결과가 달라집니다.

이산형 vs 연속형 비교

구분	연속형 점수	이산형 점수(choices)
예시	0.73, 0.81	0.0, 0.5, 1.0
Kappa 계산	어려움 (항상 "불일치")	쉬움 (범주 일치 여부)
해석	복잡	명확
openevals 설정	choices 미설정	choices=[0.0, 0.5, 1.0]

왜 0.5 구간을 두나요?

점수	의미	예시
1.0	완전히 정확	참조 답변의 핵심 내용을 모두 포함
0.5	**부분 정확**	**일부 핵심은 맞지만 중요한 내용 누락**
0.0	부정확	잘못된 정보 또는 무관한 답변

💡 **팁:** 0.5 점수 구간은 Judge가 "애매한 답변"을 억지로 0 또는 1로 분류하지 않게 해줍니다. 단, 0.5 기준이 불명확하면 Judge마다 이 구간에서 불일치가 커집니다.

공통 평가 프롬프트 설계 원칙

모든 Judge에게 동일한 기준을 주는 것이 핵심입니다.

💡 **쉽게 말하면:** 세 명의 심사위원에게 채점 기준표를 나눠주지 않으면 각자 다른 기준으로 채점합니다. LLM-as-Judge도 마찬가지 — **동일한 프롬프트 + 동일한 채점 기준**을 줘야 일치도가 높아집니다.

좋은 평가 프롬프트의 3가지 요소

명확한 기준

❌ "답변이 좋으면 높은 점수를 주세요"

✅ "핵심 내용을 **모두** 포함하면 1.0점"

이산형 점수 제한

❌ "0~1 사이 점수를 주세요"

✅ "반드시 1.0, 0.5, 0.0 중 하나만 선택"

입력 구조 명시

❌ 자유 형식

✅ <질문>, <생성된 답변>, <참조 답변> 태그

실습 프롬프트 구조 예시

프롬프트 구조

```
당신은 RAG 시스템의 답변 정확성을 평가하는 전문가입니다.

<질문> {inputs}
<생성된 답변> {outputs}
<참조 답변> {reference_outputs}

다음 기준에 따라 반드시 아래 점수 중 하나를 선택하세요:
- 1.0: 생성된 답변이 참조 답변과 동일하거나
      모든 핵심 내용을 포함하고 사실적으로 정확함
- 0.5: 일부 핵심 내용을 포함하지만
      일부 누락되거나 부정확함
- 0.0: 참조 답변과 다르거나 사실적으로 부정확함
```

구조 분석

역할 지정

"전문가입니다" → Judge LLM에게 채점자 역할을 명확히 부여

XML 태그 구분

<질문>, <생성된 답변>, <참조 답변> 태그로 각 입력을 명확히 분리

이산형 점수 강제

"반드시 아래 점수 중 하나를 선택" → choices=[0.0, 0.5, 1.0]과 연동

💡 XML 태그를 쓰는 이유: <질문>, <생성된 답변> 같은 태그는 Judge LLM이 각 부분을 정확히 구분하게 합니다. 태그 없이 긴 텍스트를 나열하면 LLM이 어느 부분을 채점해야 할지 혼란스러워합니다.

3개 Judge 모델 선택 전략

서로 다른 제조사/크기의 LLM을 Judge로 사용하는 이유

1	2	3
judge_gpt_mini	**judge_gpt_nano**	**judge_gemini**
gpt-4.1-mini | OpenAI	**gpt-4.1-nano** | OpenAI	**gemini-2.5-flash-lite** | Google
경량 + 빠름 + 비용 낮음	초경량 + 가장 빠름 + 가장 저렴	다른 아키텍처 + 다각도 검증

💡 왜 다른 제조사를 섞나요?

같은 회사(OpenAI)의 모델끼리만 비교하면 두 모델이 비슷한 편향을 가질 수 있습니다. "OpenAI 모델은 영어 답변에 더 관대하다" 같은 경향이 있다면, mini와 nano 둘 다 같은 방향으로 틀릴 수 있습니다. 다른 회사(Google) 모델을 추가하면 이런 편향을 잡아낼 수 있습니다.

Judge 선택 시 고려사항

비용·속도·정확도·다양성을 균형 있게 고려합니다.

고려사항 비교

고려사항	설명
비용	Judge 모델 API 호출 비용 (nano < mini < GPT-4o)
속도	테스트셋 전체 평가 시간 (nano가 가장 빠름)
정확도	더 큰 모델이 더 정확한 채점을 할 가능성
다양성	동일 회사 모델끼리는 편향이 공유될 수 있음

실전 선택 가이드

빠른 프로토타입

nano 2개 + 다른 제조사 1개 조합으로 비용 최소화

정확도 우선

GPT-4o 또는 Claude 같은 대형 모델을 Judge로 사용

최종 검증

Kappa로 실제 측정해서 확인하는 것이 최선

⚠️ **주의:** Judge 모델이 클수록 더 정확하다고 단정할 수 없습니다. 때로는 작은 Judge가 특정 도메인에서 더 일관된 채점을 하기도 합니다. Kappa로 실제 측정해서 확인하는 것이 최선입니다.

불일치 사례 분석 — 왜 중요한가?

Judge 간 점수 차이가 큰 사례를 분석하면 평가 시스템의 약점을 찾을 수 있습니다.

💡 **쉽게 말하면:** 세 명의 심사위원 중 한 명만 "나쁘다"고 평가한 사례가 있다면, 그 사례에서 **무엇이 모호했는지** 분석하면 채점 기준을 더 명확하게 만들 수 있습니다.

불일치 유형 3가지

유형	정의	심각도
완전 일치	모든 Judge가 동일한 점수	정상
부분 불일치	일부 Judge만 다른 점수 (범위 ≤ 0.5)	주의
완전 불일치	최대 - 최소 차이 ≥ 1.0 (예: 0.0 vs 1.0)	심각

불일치 지표 계산

```
df_answers["score_std"] = df_answers[judge_names].std(axis=1)
df_answers["score_range"] = (
    df_answers[judge_names].max(axis=1)
    - df_answers[judge_names].min(axis=1)
)
```

지표	의미	특징
score_std	점수가 평균에서 얼마나 퍼져 있나	극단적 값에 덜 민감
score_range	최댓값과 최솟값의 차이	극단적 불일치 바로 포착

이번 실습에서는 score_range로 상위 5개 불일치 사례를 추출합니다.

불일치 원인 분류 및 해결책

불일치가 발생하는 원인을 파악하면 평가 시스템을 개선할 수 있습니다.

불일치 원인 분류 예시

원인	예시	해결책
프롬프트 기준 모호	"부분 정확" 기준이 불명확	0.5점 기준 구체화
답변 스타일 차이	한 모델은 구어체 선호, 다른 모델은 문어체 선호	채점 기준에 스타일 중립 명시
모델 편향	OpenAI 모델이 OpenAI 스타일 답변에 관대	다른 회사 Judge 추가
언어 품질 과대평가	문법적으로 유창하지만 사실은 틀린 답변	Faithfulness 기준 추가

분석 → 개선 사이클

불일치 사례 추출

score_range 상위 N개 사례 선별

원인 분류

프롬프트 모호 / 모델 편향 / 스타일 차이 중 어디에 해당하는지 확인

프롬프트 개선

원인에 맞는 해결책 적용 후 Kappa 재측정

💡 불일치 사례 분석은 단순히 Kappa 수치를 높이는 것이 아니라, 평가 기준 자체를 더 명확하게 만드는 과정입니다.

Kappa 히트맵 시각화 읽는 법

Kappa 행렬 히트맵으로 Judge 쌍별 일치도를 한눈에 파악합니다.

히트맵 구조

	GPT-mini	GPT-nano	Gemini
GPT-mini	1.00	0.65	0.42
GPT-nano	0.65	1.00	0.38
Gemini	0.42	0.38	1.00

위치	의미
대각선 (1.00)	자기 자신과 비교 — 항상 완벽 일치
상삼각 / 하삼각	대칭 — A vs B = B vs A
셀 색상 (초록)	높은 일치도
셀 색상 (빨간)	낮은 일치도

💡 히트맵에서 봐야 할 것

1. 어떤 두 Judge 쌍이 가장 많이 일치하나?
2. 어떤 쌍이 가장 다르게 채점하나?
3. 같은 회사(GPT-mini vs GPT-nano) vs 다른 회사(GPT vs Gemini) 비교

시각화 코드 핵심 설정

```
sns.heatmap(
  kappa_matrix.astype(float),
  annot=True,      # 셀에 숫자 표시
  fmt=".3f",       # 소수점 3자리
  cmap="RdYlGn",   # 빨강(낮음)→노랑(중간)→초록(높음)
  vmin=-0.2,       # 색상 범위 최소
  vmax=1.0,        # 색상 범위 최대
  center=0.4,      # "보통(Moderate)" 기준
)
```

center=0.4의 의미: Moderate 기준값(0.41)을 중간 색(노란색)으로 맞춰 그 이상은 초록, 이하는 빨강으로 직관적으로 표시합니다.

가중 Kappa (Weighted Kappa)

일치하지 않더라도 "얼마나 다른지"에 따라 다르게 패널티를 줍니다.

💡 쉽게 말하면

일반 Kappa: 0.0 vs 0.5 불일치 = 0.0 vs 1.0 불일치 (모두 동일하게 "불일치")

가중 Kappa: 0.0 vs 0.5 불일치 < 0.0 vs 1.0 불일치 (거리가 먼 불일치에 더 큰 패널티)

가중 방식 비교

방식	수식 (불일치 가중치)	특징
일반 Kappa	불일치 = 모두 1	거리 무시
Linear Weights	거리에 비례	1칸 차이는 1 패널티, 2칸 차이는 2 패널티
Quadratic Weights	거리 제곱에 비례	1칸 = 1, 2칸 = 4 (큰 불일치에 훨씬 큰 패널티)

구체적인 예시 (점수 3단계: 0.0 / 0.5 / 1.0)

Judge A	Judge B	일반 Kappa 패널티	Quadratic 패널티
1.0	0.5	1 (불일치)	1 (1칸 차이^2)
1.0	0.0	1 (불일치)	4 (2칸 차이^2)

가중 Kappa – 코드와 사용 기준

언제 가중 Kappa를 써야 하는지 판단하는 기준입니다.

코드

```
# 일반 Kappa
kappa_normal = cohen_kappa_score(labels_a, labels_b)

# Quadratic 가중 Kappa
kappa_weighted = cohen_kappa_score(
    labels_a, labels_b, weights="quadratic"
)

print(f"일반 Kappa:  {kappa_normal:.4f}")
print(f"가중 Kappa: {kappa_weighted:.4f}")
```

언제 가중 Kappa를 쓰나요?

상황	권장 방식
0.0 vs 1.0 불일치가 특히 심각한 경우	Quadratic Weighted Kappa
모든 불일치를 동등하게 처리	일반 Kappa
순서 없는 범주 (예: 언어 종류)	가중 Kappa 사용 불가

💡 **이번 실습의 점수(0.0 / 0.5 / 1.0)는 순서가 있는 범주이므로 가중 Kappa가 의미 있습니다.** 0.0 vs 1.0 불일치(완전 반대)가 0.0 vs 0.5 불일치보다 훨씬 심각하기 때문입니다.

앙상블 Judge — 신뢰도 향상 전략

여러 Judge의 점수를 합산해 단일 Judge보다 안정적인 판정을 내립니다.

💡 **쉽게 말하면:** 축구 심판 3명이 "오프사이드냐 아니냐"를 판정할 때, 3명 중 2명 이상이 "오프사이드"라고 하면 오프사이드로 결정하는 것이 **다수결(Majority Voting)**입니다. LLM-as-Judge에서도 Judge 3개 중 2개가 같은 점수를 주면 그 점수를 최종 판정으로 사용합니다.

앙상블 방식 비교

방식	수식	특징
다수결 (Majority Vote)	가장 많이 나온 점수	극단적 판정 하나를 무력화
평균 (Mean)	점수 합계 / Judge 수	부드러운 합산, 해석 쉬움
가중 평균 (Weighted Mean)	Σ(점수 × 신뢰도) / Σ 신뢰도	더 신뢰도 높은 Judge에 더 큰 가중치

앙상블 Judge의 효과

상황	단일 Judge	앙상블 Judge
A: 1.0, B: 1.0, C: 0.0	선택에 따라 다름	1.0 (다수결 2:1)
A: 0.0, B: 0.5, C: 1.0	선택에 따라 다름	0.5 또는 평균 0.5
모두 동의	동일	동일

앙상블 Judge — 다수결 구현 코드

Python으로 다수결 앙상블을 직접 구현하는 방법입니다.

다수결 구현 코드

```
from collections import Counter

def ensemble_judge(scores_dict):
    """
    scores_dict: {"GPT-mini": 1.0, "GPT-nano": 0.5, "Gemini": 1.0}
    → 다수결로 최종 점수 반환
    """
    all_scores = list(scores_dict.values())
    counter = Counter(all_scores)
    return counter.most_common(1)[0][0]

# 예시
scores = {"GPT-mini": 1.0, "GPT-nano": 0.5, "Gemini": 1.0}
final = ensemble_judge(scores)  # → 1.0 (2/3 다수결)
```

언제 앙상블이 특히 유용한가요?

→ **이상 점수 방어:**

한 Judge가 특정 질문 유형에서 지속적으로 이상한 점수를 줄 때

→ **비용 최적화:**

비용 절감을 위해 저렴한 Judge를 여러 개 조합할 때

→ **장애 대비:**

단일 Judge의 API 장애 시 대체 수단 확보

평가 신뢰도 향상 전략 종합

Kappa가 낮게 나왔을 때 개선하는 방법들

진단 → 해결 로드맵

Kappa 계산

- κ ≥ 0.61 → 양호 ✓ (현 시스템 유지)
- 0.41 ≤ κ < 0.61 → 보통 (아래 개선책 1~2개 시도)
- κ < 0.41 → 개선 필요 (아래 개선책 전체 검토)

개선책 목록

개선책	방법	기대 효과
프롬프트 명확화	0.5점 기준을 구체적 예시로 설명	0.5점 구간 불일치 감소
Judge 모델 교체	더 큰/다른 모델 사용	일관성 향상
앙상블 Judge	3개 Judge 다수결	극단적 불일치 무력화
Few-shot 예시 추가	프롬프트에 채점 예시 3개 추가	Judge가 기준을 더 정확히 이해
평가 기준 단순화	3단계(0/0.5/1) → 2단계(0/1)	판단 모호성 제거